

	SOFTWARE DESIGN DESCRIPTIONS OF Spacecraft Status Display and Commanding Software 1.0	Doc. No: SCG-GRCD2-SW-2021-02 Version : 1.0
---	--	--

Software Design Descriptions

Spacecraft Status Display and Commanding Software (SPASDACS) 1.0

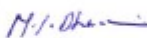
Geosat RF & Payload Checkout Division-2
Spacecraft Checkout Group

AUGUST 2021
UR RAO SATELLITE CENTRE
INDIAN SPACE RESEARCH ORGANISATION
BANGALORE

	SOFTWARE DESIGN DESCRIPTIONS OF Spacecraft Status Display and Commanding Software 1.0	Doc. No: SCG-GRCD2-SW-2021-02 Version : 1.0
--	--	--

	SOFTWARE DESIGN DESCRIPTIONS OF Spacecraft Status Display and Commanding Software 1.0	Doc. No: SCG-GRCD2-SW-2021-02 Version: 1.0
---	--	---

AUTHENTICATION

	Name	Signature	Date
Prepared By	Suraj Bhosle		17/08/2021
	Janardhana Reddy G		17/08/2021
Reviewed & Approved By	M.S. Bhuvaneswari		17/8/2021

URSC QUALITY POLICY: COMMITTED TO TOTAL QUALITY AND ZERO DEFECT IN SPACE SYSTEMS AND SERVICES

URSC QUALITY POLICY: COMMITTED TO TOTAL QUALITY AND ZERO DEFECT IN SPACE SYSTEMS AND SERVICES



SOFTWARE DESIGN DESCRIPTIONS OF
Spacecraft Status Display and Commanding Software 1.0

Doc. No: SCG-GRCD2-SW-2021-02
Version : 1.0

Change History

Version Number	Date	Affected Section, Figure, Table	Nature of change [A,M,D] *	Description
1.0	02/08/2021	-	New	

* A – Addition, D - Deletion, M – Modification



Table of Contents

1	INTRODUCTION	8
1.1	PURPOSE	8
1.2	SCOPE	9
1.3	OVERVIEW	9
1.4	REFERENCE MATERIAL	9
1.5	DEFINITIONS, ACRONYMS & ABBREVIATIONS	9
1.5.1	Acronyms & Abbreviations	9
2	SYSTEM OVERVIEW	11
2.1	SYSTEM FUNCTIONALITY	11
2.2	DESIGN OF SYSTEM	11
3	SYSTEM ARCHITECTURE	12
3.1	ARCHITECTURAL DESIGN	12
4	SOFTWARE DESIGN DESCRIPTION	14
4.1	DECOMPOSITION DESCRIPTION	14
4.1.1	Use case Diagram.....	15
4.1.2	Sequence diagrams.....	16
4.1.3	Class Diagram.....	18
4.2	DESIGN RATIONALE.....	18
5	DATA DESIGN.....	19
5.1	DATA DESCRIPTION.....	19
5.2	DATA DICTIONARY	19
6	COMPONENT DESIGN	20
6.1	APP BUILDER	21
6.1.1	App Builder Class Diagram	24
6.1.2	JavaScript code generation process for elements	24
6.1.3	Predefined JavaScript code files	25
6.2	USER AUTHENTICATION MODULE	27



6.3	TM/SMON/ADC MAPPING DATA STORE/RETRIEVE MODULE	29
6.4	VISUAL STUDIO CODE EDITOR CONFIGURATION MODULE	29
6.5	TM/SMON/ADC SIMULATOR.....	30
6.6	CLIENT-SIDE JAVASCRIPT	31
6.6.1	UI Thread functionality	31
6.6.2	TM/SMON/ADC Data Reception and pre-processing by Web Worker	31
6.7	INKSCAPE EDITOR	34
6.7.1	Symbol library	35
6.8	DETAILED DESIGN DESCRIPTION	35
	<i>API Reference</i>	35
	<i>Subpackages</i>	35
	src.Core.EditorDataAPI	35
	src.Core.database.attributeStore	37
	src.Core.database.getMnemonics.....	39
	src.Core.svgComponents.SVG_Parser	40
	src.Core.svgComponents.componentFactory	41
	src.Core.svgComponents.component_customComponent.....	42
	src.Core.svgComponents.component_userComponent	44
	src.Core.svgComponents.replaceBlocksBase.....	45
	src.Core.utility.colors.....	49
	src.Core.utility.generalFunctions	49
	src.Core.utility.getDropDownTm.....	50
	src.Core.utility.registerSvgNameSpace	51
	src.Core.API.....	51
	src.Core.Main.....	52
	src.Core.WriteHtml.....	55
	src.Core.logger_config	57
	src.Core.login_module	58
	src.Core.writeCss.....	60
	src.Core.writeJavaScript.....	61
6.9	INTERFACE WITH DATABASE.....	64



6.10	INTERFACE WITH SPACECRAFT CHECKOUT COMPUTER (SCC).....	64
7	HUMAN INTERFACE DESIGN	65
7.1	OVERVIEW OF USER INTERFACE.....	65
7.2	SCREEN IMAGES	66
8	REQUIREMENTS MATRIX	68



List of Tables

Table 1: Reference Documents	9
Table 2: Acronyms & Abbreviations	10
Table 3: TM/SMON Mnemonics renaming examples	33
Table 4 Requirements Matrix	68

List of Figures

Figure 1: SPASDACS GUI prototype	8
Figure 1 Top level interface architectural design of SPASDACS.....	12
Figure 2: Use Case Diagram.....	15
Figure 3: Normal Users Role Sequence Diagram	16
Figure 4: Project Managers role Sequence Diagram	17
Figure 5: Class Diagram of SPASDACS Application.....	18
Figure 6: Excel sheet format.....	19
Figure 7 : Build Flowchart.....	21
Figure 8: Event Handler flowchart	23
Figure 9: App Builder Class Diagram	24
Figure 10: Code generation Process for general elements	25
Figure 11: VS code editor popup for element	26
Figure 12: JavaScript code for assigning these attributes to elements	27
Figure 13 : User Authentication Module.....	28
Figure 14: VS code editor for mapping element attributes to mnemonics	29
Figure 15 : Simulator Flowchart.....	30
Figure 16: SPASDACS GUI-1	66
Figure 17: SPASDACS GUI-2	66
Figure 18: Dropdown VS code Editor for assigning TM/TC.....	67
Figure 19: Mnemonic dropdown inside VS code editor.....	67

1 Introduction

1.1 Purpose

Spacecraft Status Display and Commanding Software is for Real Time display of an ‘Active Block diagram’ of a spacecraft subsystem for Pictorial display of Telemetry/SMON/ADC data Status.

This software gives pictorial representation of spacecraft telemetry instead of text display. Pictorial display are more intuitive then text.

Software Name: Spacecraft Status Display and Commanding Software 1.0

As shown in GUI prototype Fig.1 below,

This software shall enable user to draw system block diagram and allows changing its visual appearance based on live data (telemetry/SMON/ADC data).

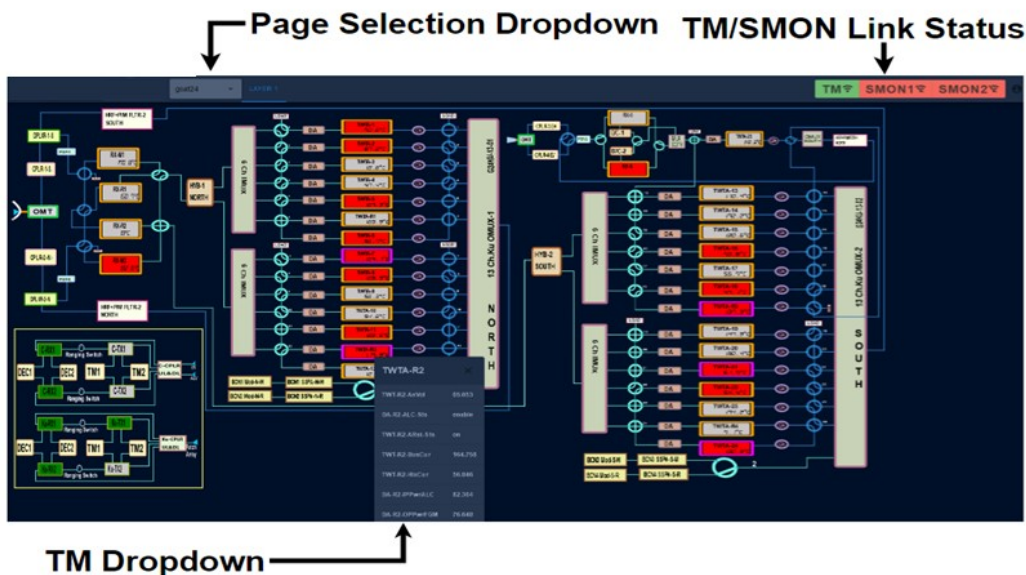


Figure 1: SPASDACS GUI prototype

1. It should provide alerts (audio & visual) on invalid / prohibitive combinations defined by the user.
2. Provision should be there to Conditional Hide/Display of a block.
3. Should indicate Test path/Signal flow path.



4. On Mouseover of block TM should be visible.
5. Provision should be there to select different pages
6. Provision shall be there to indicate telemetry/SMON/ADC link status.
7. Built in Simulator should be there to test the created block diagram.

To realize and implement all the requirements; Application is developed in python, which generates client side web application code. This document is generated on the guidelines of ISPD2.

1.2 Scope

This document describes the design details of SPASDACS, to realize all the requirements listed in the 'SRS for SPASDACS software. This document is meant to aid in the implementation, test and evaluation and the operations of this software in its intended target environment.

1.3 Overview

This document describes the design of the SPASDACS software.

Section 1: Introduction

Section 2: System overview: Describes functionality, context and design.

Section 3: System Architecture: Describes architectural design, decomposition description & design rationale

Section 4: Data design: Describes data description and database.

Section 5: Component design: Describes design of each component in detail.

Section 6: Human interface Design: Describes overview of user interface.

Section 7: Requirements Matrix: lists the Traceability matrix of the requirements as specified in the SRS document.

1.4 Reference Material

Document Name	Document Number	Publishing Date
SOFTWARE REQUIREMENTS SPECIFICATION FOR SPASDACS Software	SCG-GRCD2-SW-2021-01	Aug 2021

Table 1: Reference Documents

1.5 Definitions, Acronyms & Abbreviations

1.5.1 Acronyms & Abbreviations

The table below gives the definition for the abbreviations as used in this document



S. No	Acronym / Abbreviation	Definition
1.	IST	Integrated Satellite Test
2.	PASDACS	Payload status display and commanding software.
3.	SPASDACS	Spacecraft status display and commanding software
4.	GUI	Graphical User Interface
5.	SCC	Spacecraft Checkout Computer
6.	COMPACSS	Communication Payload Checkout Software System
7.	API	Application Programming Interface
8.	SOA	Service Oriented Architectures
9.	WAMP	Web Application Messaging Protocol
10.	JSON	JavaScript Object Notation
11.	TM	Telemetry
12.	TC	Telecommand
13.	CSS	Cascading Style Sheets
14.	HTML	Hypertext Markup Language
15.	SVG	Scalable Vector Graphics
16.	FTP	File Transfer Protocol
17.	UDP	User Datagram Protocol
18.	SCOS	Special Checkout System
19.	SMON	SCOS Monitoring
20.	ADC	Analog to Digital Converter
21.	RPC	Remote Procedure Call

Table 2: Acronyms & Abbreviations



2 System Overview

2.1 System functionality

This software product will be used for Real Time display of an 'Active Block diagram' of a spacecraft subsystem for Pictorial display of Telemetry/SMON/ADC data Status.

2.2 Design of system

For developing this software Micro-services architecture is followed. Micro- services are a specialization of an implementation approach for service-oriented architectures (SOA) used to build flexible, independently deployable software systems.

Following software/frameworks are used to develop the system

- Inkscape editor for diagram drawing. It is an open source svg editor using this user can create/edit svg images.
- A python application as a micro service for building/ deploying web application.
- Nodejs is used for server side rendering.
- Vuejs, Vuetify, GSAP, Snapsvg, animejs framework/libraries are used in front end.

3 System Architecture

3.1 Architectural Design

Following diagram describes high level system interface architecture

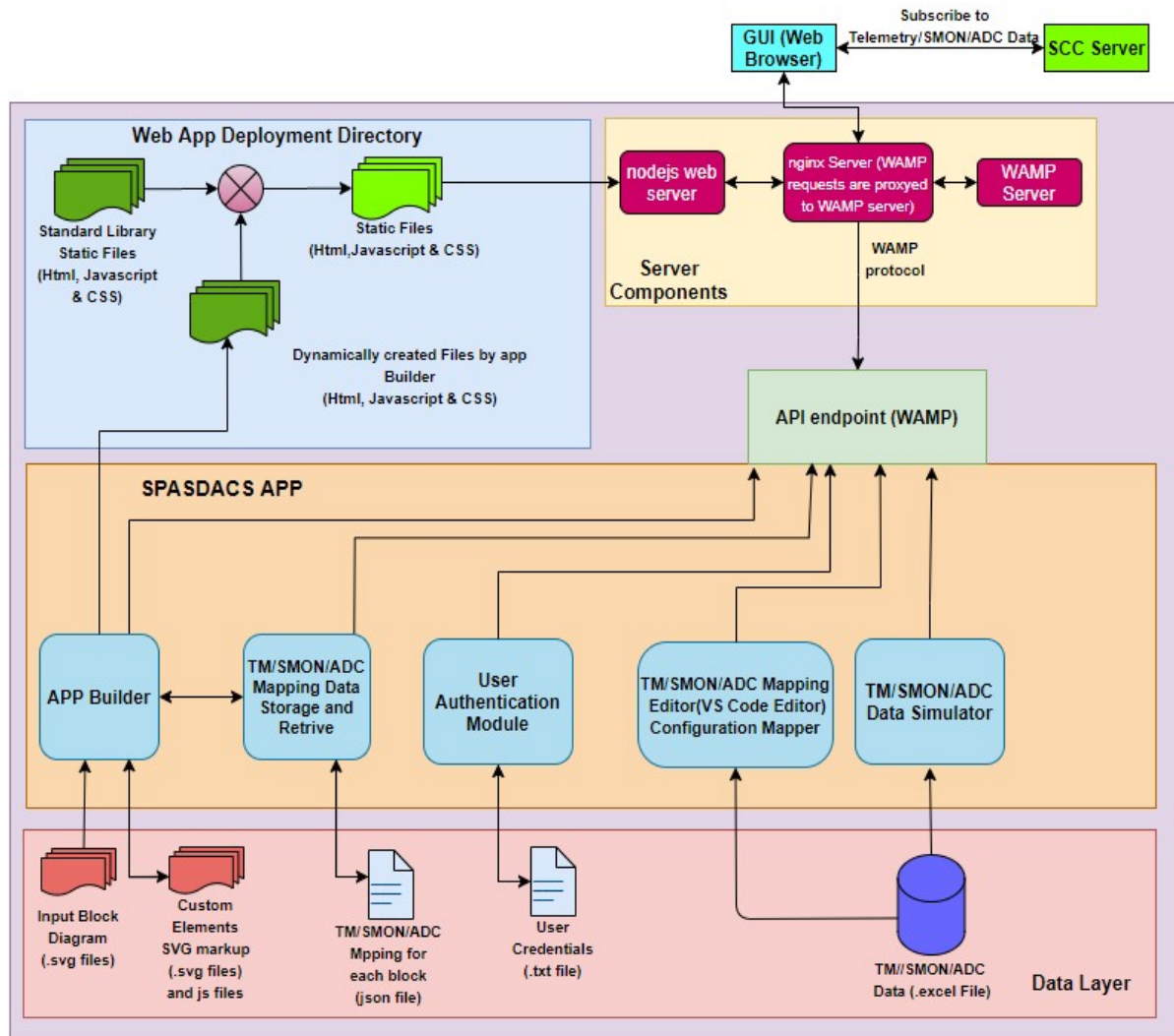


Figure 2 Top level interface architectural design of SPASDACS

SPASDACS is a web-based application for displaying spacecraft subsystems status in pictorial form. SPASDACS app is consisting of app builder (which creates front end application code), TM/SMON/ADC data simulator, TM/SMON/ADC data mapping editor, and User authentication module. SPASDACS application will communicate with GUI (web browser) through Web Server using WAMP protocol.



User has to make block diagram of spacecraft systems using inkscape editor. For more frequently used elements like TWTA, switches, Wheels etc. symbol library is created and included with inkscape editor (refer section 6.7). Once diagrams are ready and stored in input directory user has to open GUI (browser) and should initiate build, the browser will call App builder module of SPASDACS through RPC using WAMP protocol. SPASDACS App builder will process the input files and generates HTML, JavaScript & CSS files and stores the files in web application deployment directory. After build completion browser auto reload the page.

User has to assign TM/SMON/ADC data mapping for all elements. This data mapping for each element is stored in json file. When user build application after data mapping, App Builder will take block diagram and mapping data json file as input and dynamically it will create HTML, JavaScript & CSS files.

When user runs the SPASDACS and loads page on web browser, subscription request for Telemetry/SMON/ADC data will go to SCC server. After that SCC server will continuously send data to web browser.

API endpoint for SPASDACS app is through WAMP.

Telemetry/SMON/ADC data is stored in excel file, user has to populate this excel sheet with corresponding mnemonics. Details of excel file is described in Section 5.1.



4 Software Design Description

4.1 Decomposition Description

The system top level architecture is shown in figure 1. Where the GUI is developed with Vue.js and Vuetify. Web application messaging protocol (WAMP) is used to update GUI in real time and to trigger RPC.

WAMP is a web socket sub protocol, specified to offer routed RPC and PubSub. Its design goal is to provide an open standard for soft real time message exchange between application components and ease the creation of loosely coupled architectures based on micro services. Because of this, it is a suitable enterprise service bus (ESB), fit for developing responsive web applications or to coordinate multiple connected devices in IOT.

Here the GUI code (HTML, CSS, and JavaScript) is generated dynamically by SPASDACS app builder. App builder will take user diagrams (.svg) and data mapping data as input and generates front end code, which will be served by nodejs server.

Vue.js is used as client-side JavaScript framework. Vue.js is an open-source progressive JavaScript framework for building user interfaces. Integration into projects that use other JavaScript libraries is made easy with Vue because it is designed to be incrementally adoptable.

Node.js is an open source, cross-platform JavaScript run-time environment for executing JavaScript code server-side. Node.js enables JavaScript to be used for server-side scripting, and runs scripts server-side to produce dynamic web page content before the page is sent to the user's web browser.

4.1.1 Use case Diagram

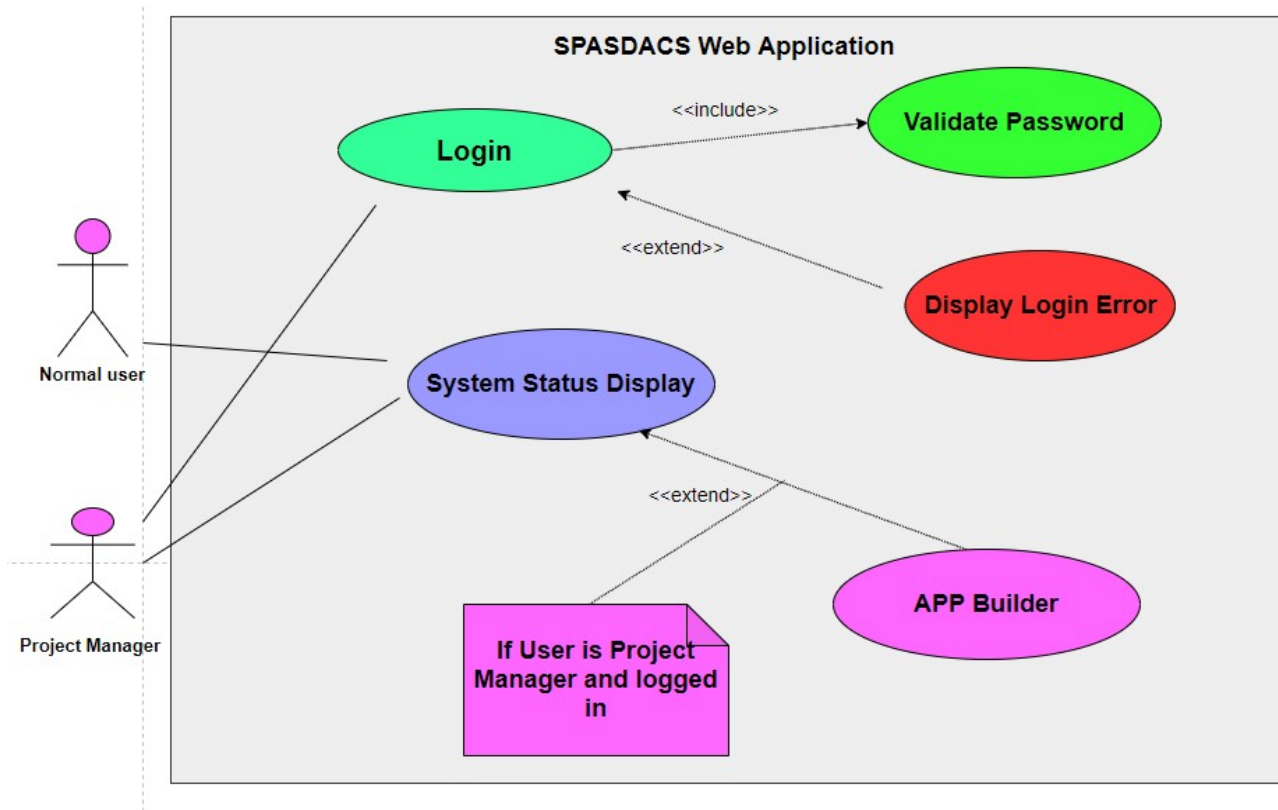


Figure 3: Use Case Diagram

As shown in use case diagram, two types of user roles are allowed,

- Normal User
- Project Manager

The application has two modes of operation based on user role the corresponding role will get activated as explained below.

1. **TM view mode:** - In this mode user can see the live telemetry in SPASDACS. This mode of operation is allowed to every user for this mode login credentials not required.
2. **Programming mode:** - In this mode previous mode features are allowed and also user can edit block diagram, can assign telemetry/smon/adc data to visual elements. Only Project Manager Role is allowed to operate SPASDACS in this mode.

4.1.2 Sequence diagrams

The Sequence diagram for Normal User role is shown below.

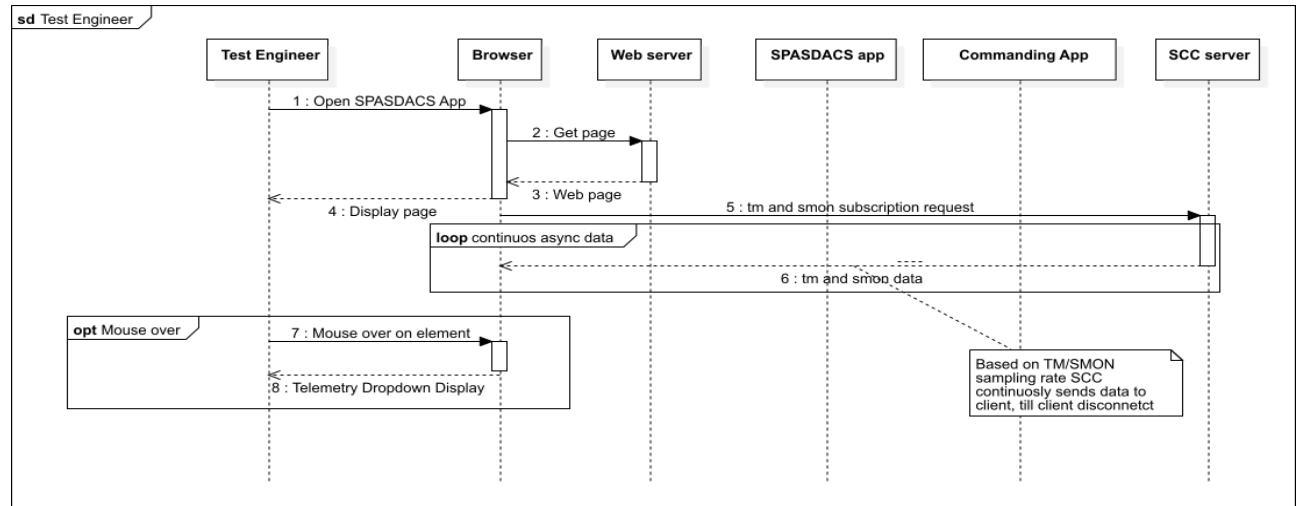


Figure 4: Normal Users Role Sequence Diagram

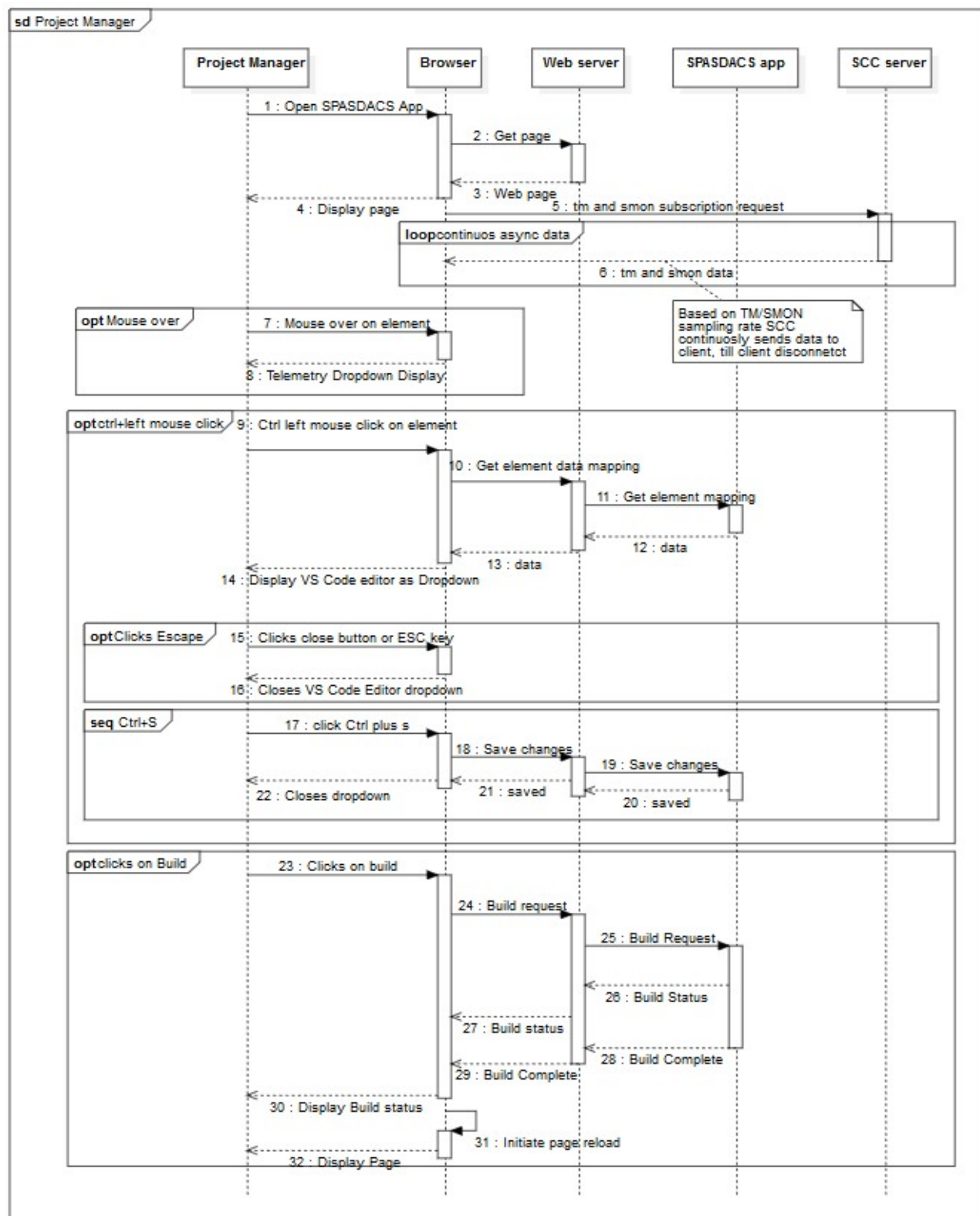


Figure 5: Project Managers role Sequence Diagram

4.1.3 Class Diagram

The overall applications class diagram is shown below. For detailed explanation of classes refer section 6.8.

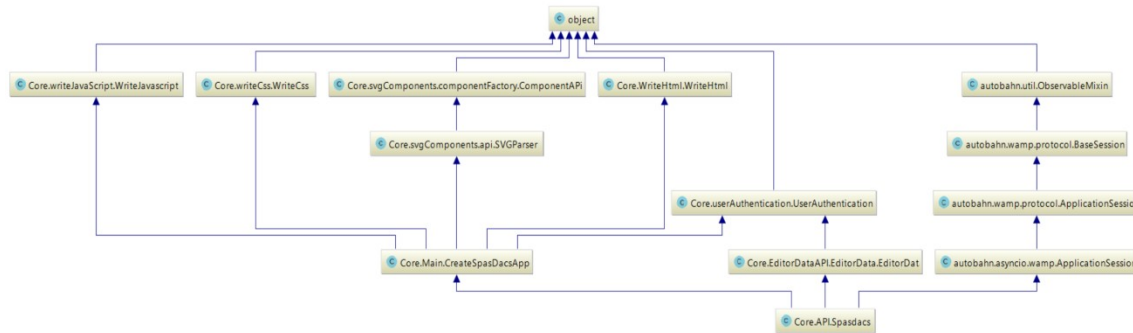


Figure 6: Class Diagram of SPASDACS Application

4.2 Design Rationale

In micro-services architecture, services should have a small granularity and the protocols should be lightweight. A central micro-services property that appears in multiple definitions is that services should be independently deployable. The benefit of distributing different responsibilities of the system into different smaller services is that it enhances the cohesion and decreases the coupling. This makes it easier to change and add functions and qualities to the system at any time. It also allows the architecture of an individual service to emerge through continuous refactoring, and hence reduces the need for a big up-front design and allows for releasing software early and continuously.



5 Data Design

5.1 Data Description

Diagrams will be stored in svg file format in input file directory. TM/SMON/ADC Data mapping of element is stored in json file.

This application does not require any data base, however for auto completion of mnemonics in data mapping editor and for TM/SMON/ADC simulation it will use TM/SMON/ADC mnemonics stored in excel sheet.

The format of excel sheet is as follows

	A	B	C	D	E
1	S_NO	MNEMONIC	POSSIBLE_STATES	LOWER_LIMIT_ALARM	UPPER_LIMIT_ALARM
2	1	TM-Mode	Normal,Normal		
3	2	OBT	20;5000	1	
4	3	C-TX2-Sts	on,off		
5	4	C-TX1-Sts	on,off		
6	5	TTC-SW3-POS-Sts	POS1,POS2		
7	6	TTC-SW1-POS-Sts	POS1,POS2		
8	7	KU-TTC-TX1-Sts	on,off		
9	8	KU-TTC-TX2-Sts	on,off		

Figure 7: Excel sheet format

There sheets with names SMON, TC, ADC should present. And in sheet the column names are same as shown above.

MNEMONIC: This column will contain TM/SMON/ADC mnemonics. This data is used by SPASDACS App and Simulator.

POSSIBLE_STATES: This column contains possible values for each mnemonic. For digital data it contains comma separated states and for analog data it contains lower, upper limit of the data separated by semi colon. The simulator simulates random data from these possible states. This data is used only by Simulator.

LOWER_LIMIT_ALARM & UPPER_LIMIT_ALARM: These columns are to simulate limit crossing alarms for analog parameters. A '1' in this column will trigger simulator to simulate value which is outside the parameter value range. A '0' or blank in this column will not cause any effect on simulator. This data is used only by Simulator.

This excel sheet should present inside 'inputFile' folder of the application.

5.2 Data Dictionary

This application does not require any data base,



6 Component Design

The SPASDACS Application consist following components

1. App Builder.
2. User Authentication Module.
3. TM/SMON/ADC Mapping Data Store/Retrieve Module.
4. Visual studio code Editor Configuration module.
5. TM/SMON/ADC Simulator.

6.1 App Builder

Below is the detailed flowchart of SPASDACS App builder module. When user wants to build the application, On GUI user has to click 'Build button' on menu or shortcut key 'Ctrl+a' which triggers Build request from GUI. Request will be forwarded to this module.

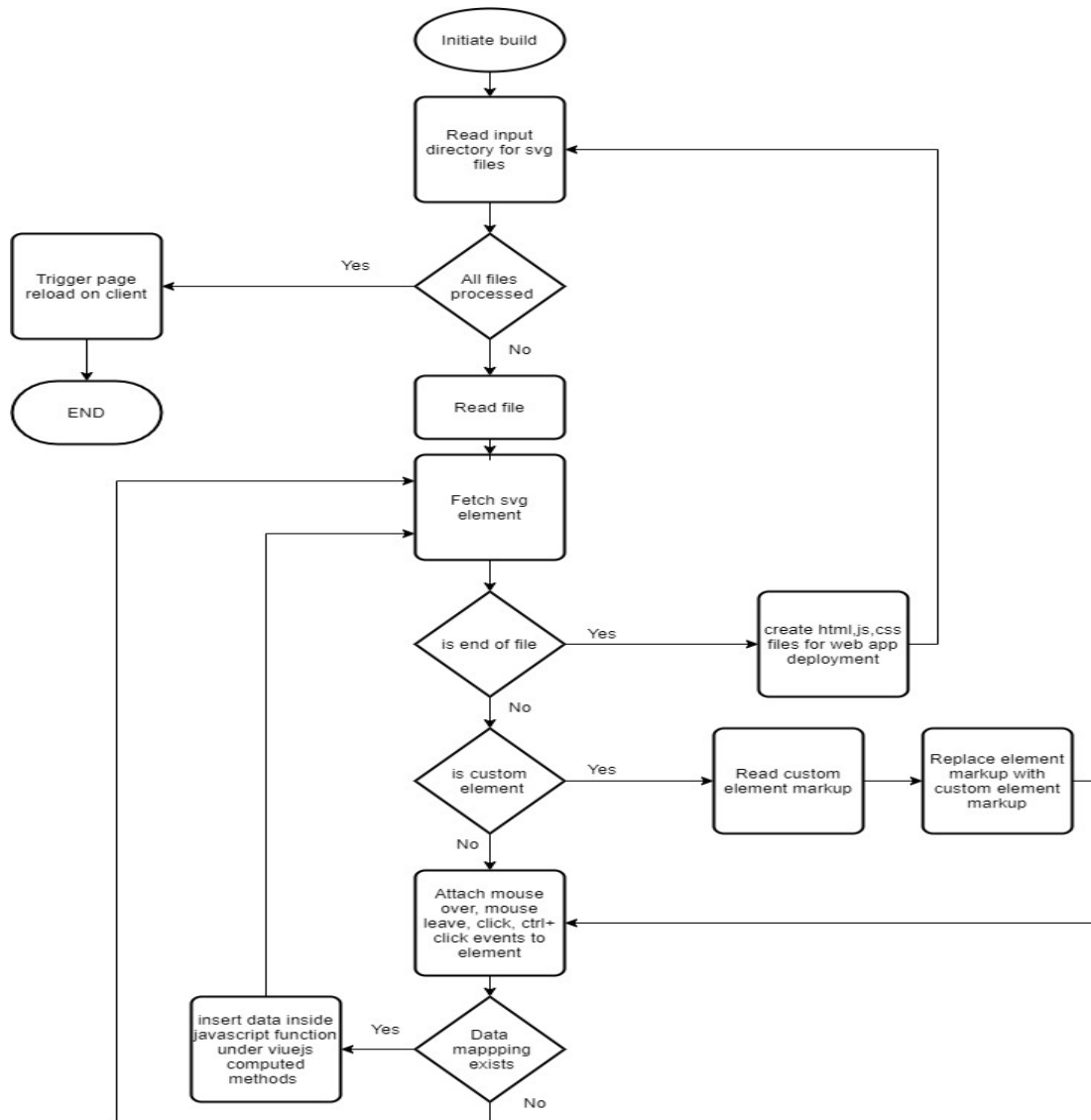


Figure 8 : Build Flowchart

When user initiates Build, this module will read input directory for svg files. It reads iteratively all the svg files in the directory. In each file it will fetch svg elements one by one until the end of file. If the fetched element is custom element, It will read custom element markup svg file (replaceBlocks.svg) in component_config_files directory and replace element mark up with custom element markup. For every svg element mouse over, mouse leave, click, ctrl+ click events will be attached. For each svg element, it checks attribute mapping defined by user if mapping



exists a JavaScript function for that element will be created inside vuejs computed methods. Finally for each svg file corresponding HTML, JavaScript and CSS file will be generated and stored in web application deployment directory.

The event handlers attached for each element is described below

1. Mouse over Event:

The event handler for this will display TM/SMON/ADC dropdown table if user had defined required dropdown mnemonics inside element attribute mapping.

2. Mouse Leave Event:

The event handler for this will close TM/SMON/ADC dropdown table.

3. Ctrl + Click Event:

The event handler for this will display VS Code editor for defining attribute mapping for the svg element. When this event triggered, RPC is triggered through WAMP for getting element attribute mapping. Remote procedure will check the mapdata.json files if mapping exists in the file, it will return the data else it will return default mapping defined for that element in component definitions.

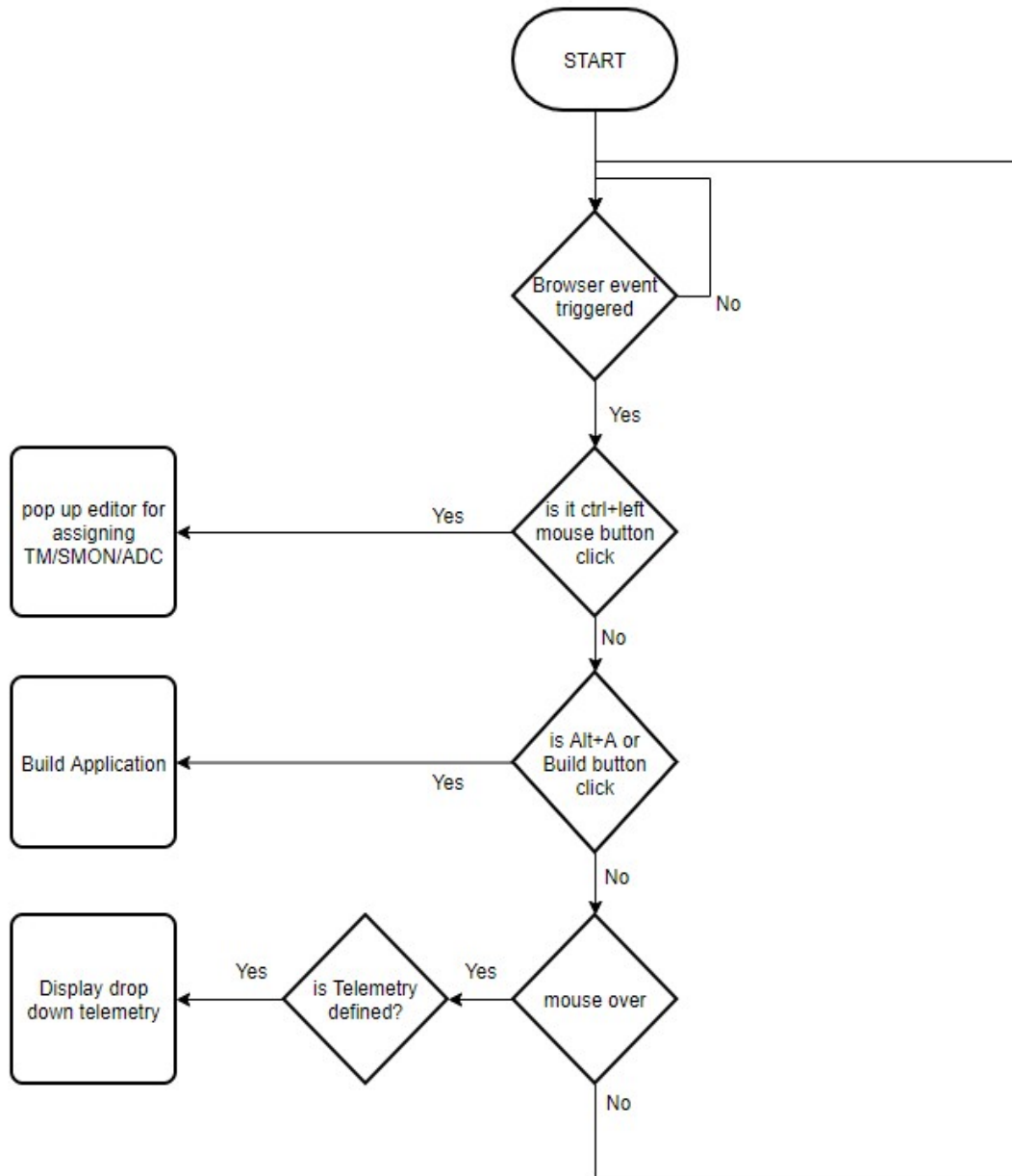


Figure 9: Event Handler flowchart

6.1.1 App Builder Class Diagram

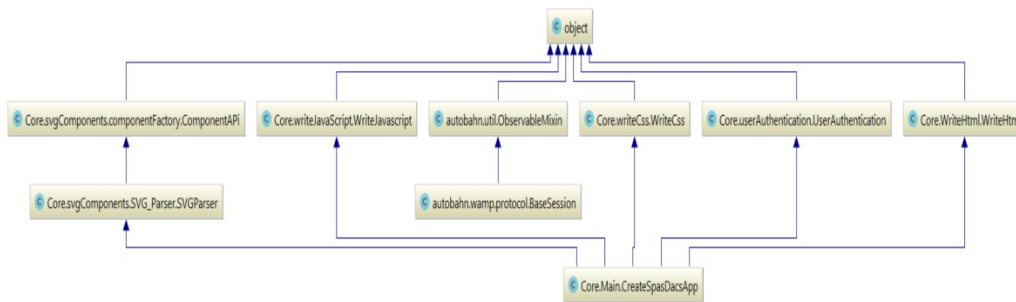


Figure 10: App Builder Class Diagram

6.1.2 JavaScript code generation process for elements

SPASDACS svg diagram have user elements and custom elements. Code generation procedure is explained below:

Software will check whether attribute mapping is defined for given element ID, if mapping exist then one JavaScript function will be created with function name as element ID, this functions will be added to vue.js computed methods for making it reactive to data changes. Initial content of this function will be element id is injected as **var elmId=id of the elemnt** and all attributes of the element as **var initial_attributes = dictionary of attributes**

Based on element ID component type will be extracted from which predefined JavaScript code for that component will be fetched from the file and this code will be appended to user defined attribute mapping. If drop down attribute is mentioned, then code will create HTML table for dropdown data and attach mouse over event to it. When mouse is hover on the element, html dropdown table will be displayed.

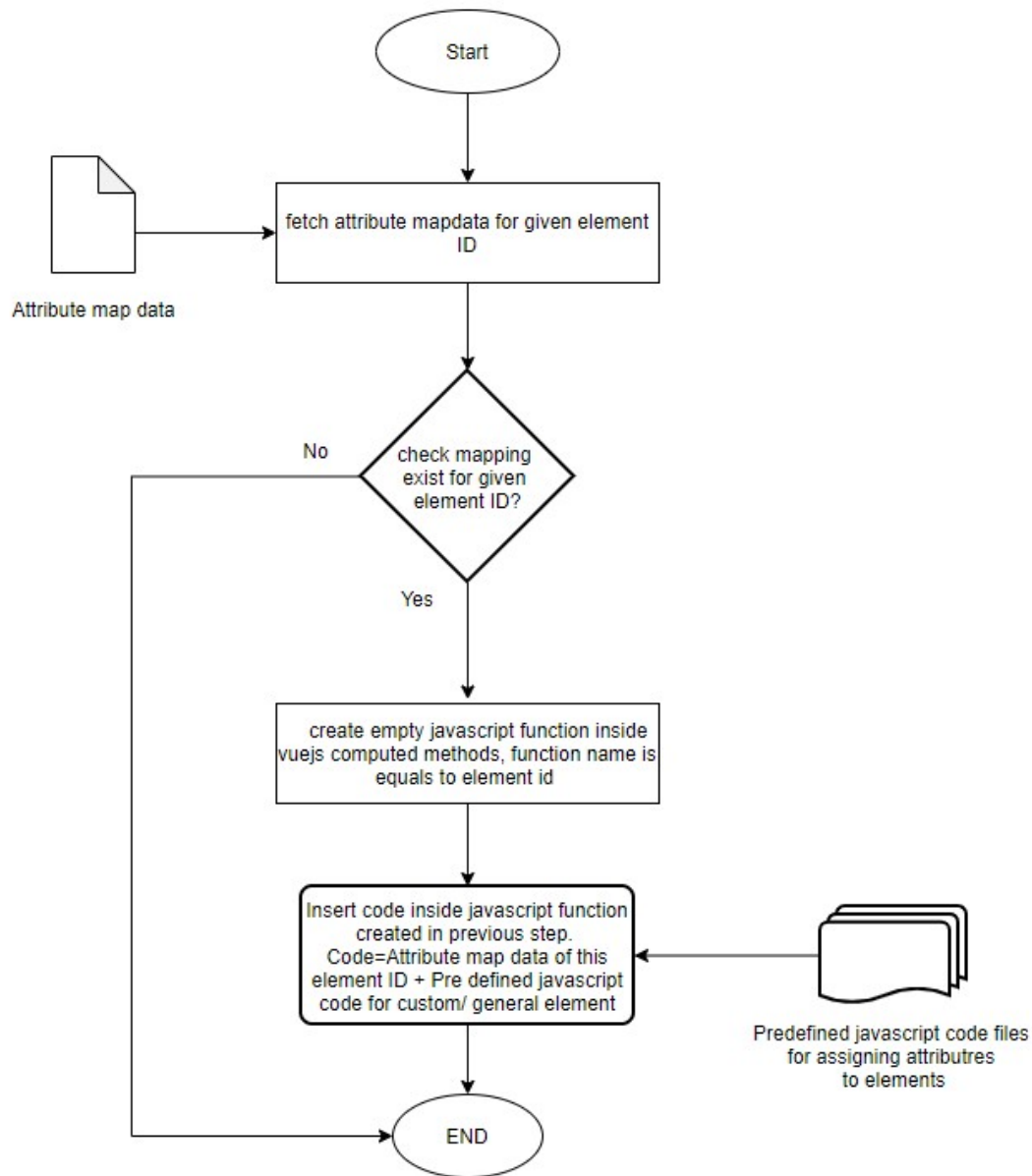


Figure 11: Code generation Process for general elements

6.1.3 Predefined JavaScript code files

For each component type two predefined JavaScript files are created by developer and will be placed inside SPASDACS “config_files” directory. One of this file is used for attributes



mapping i.e. when user generates Ctrl+Click event on GUI VS code editor will pop up with content of this file as shown below for “TWTA” component

```
TWTA__use23584 x
1  let attributes = {
2      title: "Title",
3      trunOnRequirement: (() => {
4          return false;
5      })(),
6      onCondition: (() => {
7          return false;
8      })(),
9      hlxOnCondition: (() => {
10         return false;
11     })(),
12     alcOnCondition: (() => {
13         return false;
14     })(),
15     temperature: (() => {
16         return 0;
17     })(),
18 }
19
20 let dropdown_tm={
21     title: " ",
22     tm: []
23 }
```

Figure 12: VS code editor popup for element

The second file contains JavaScript code for assigning these attributes to elements
As shown below for TWTA component.



```
1 Snap(elmId + "_title").attr({'text': attributes.title });
2 Snap(elmId + "_temperature").attr({'text': Math.round(attributes.temperature*10)/10+ "°C" });
3 let led = "#cbc6c6";
4 let e1f = "#909090";
5 let e2f = "#585756";
6 if (attributes.alcOnCondition) e1f = "rgb(255, 4, 255)";
7 if (!attributes.trunOnRequirement) { e1f = "#FFA500"; }
8 if (attributes.onCondition) {
9     led = "#34ec34";
10    if (attributes.hlxOnCondition) { led = "#00fa00"; } else { led = "#0f7442"; }
11    if (!attributes.trunOnRequirement) led = "#ff0000";
12 }
13 anime({ targets: elmId + '_edge1', fill: e1f, duration: 1500 });
14 anime({ targets: elmId + '_led', fill: led, duration: 1500 });
15
```

Figure 13: JavaScript code for assigning these attributes to elements

In the code Snap and anime js JavaScript libraries are used to assign attributes to elements and to animate.

Every svg element can have following attributes

1. Fill (background-color).
2. Stroke (border color).
3. Stroke-width (border width).
4. Position (x & y)
5. Width
6. Height.
7. Rotation.
8. Visibility.

All these attributes can be changed/ animated by using open source libraries anime/Snap/gsap.

6.2 User Authentication Module

Below is the detailed flowchart of User authentication module. After installation of SPADACS, during first login user has to register himself as admin. Only one admin account is allowed. Admin is allowed to register new users

The login/signup request from user will be forwarded to this module.

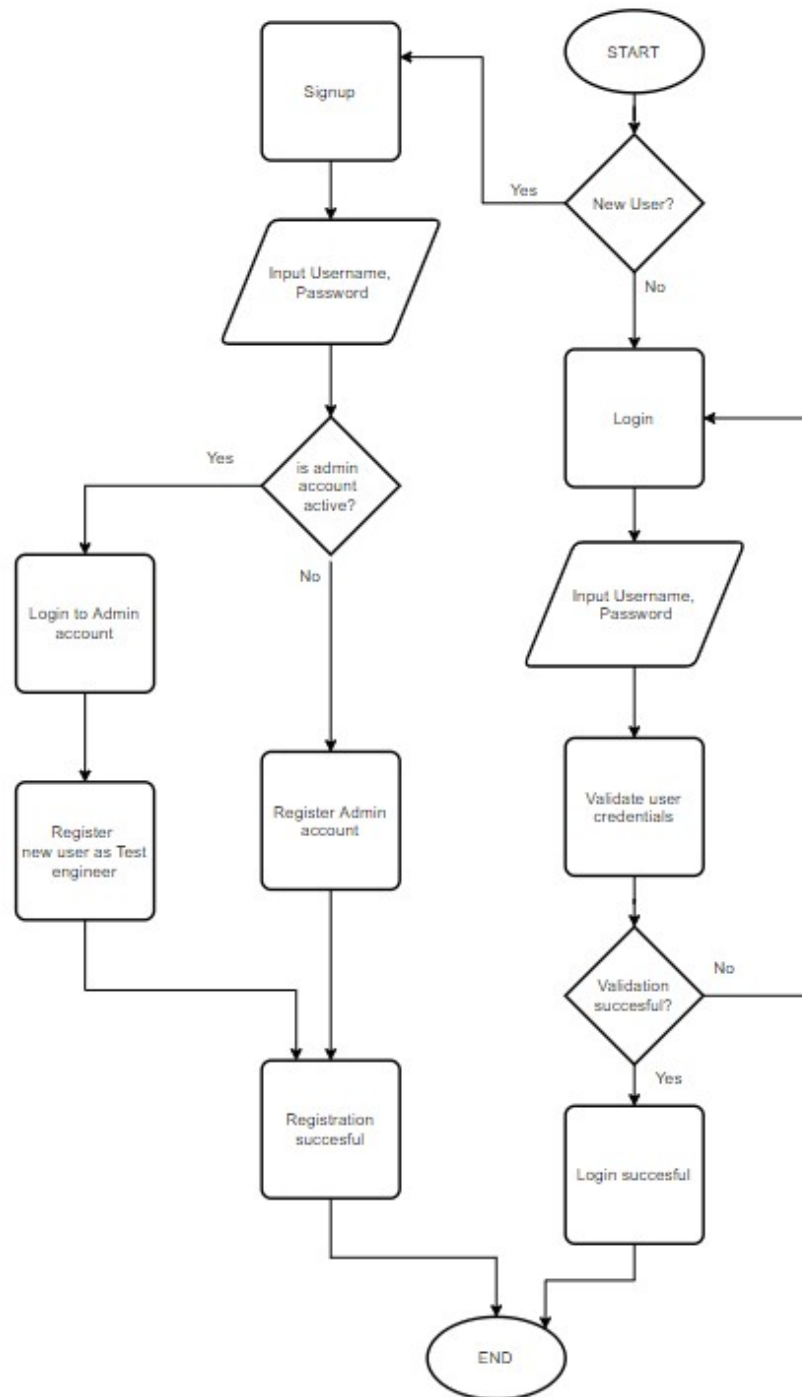


Figure 14 : User Authentication Module



If the request is for login, this module validates the credentials and sends user type to UI. In case of signup request, module will check whether admin account exists or not. If admin account exists then it will request admin credentials to register new user. Otherwise it will register new user as admin.

6.3 TM/SMON/ADC Mapping Data Store/Retrieve Module

This Module is responsible for storing and retrieving attribute mapping data from json file. When user press ctrl+click on any visual element, RPC request will be made to this module, based on element id, this modules checks the mapping in the json file if mapping exists it will return the data otherwise it will fetch default mapping data from component class.

6.4 Visual studio code Editor Configuration module

VS Code editor is used for mapping element attributes to mnemonics. For this VS code editor to be configured for JavaScript syntax and for auto completion of mnemonics. All these mnemonics to be added in VS code configuration file.

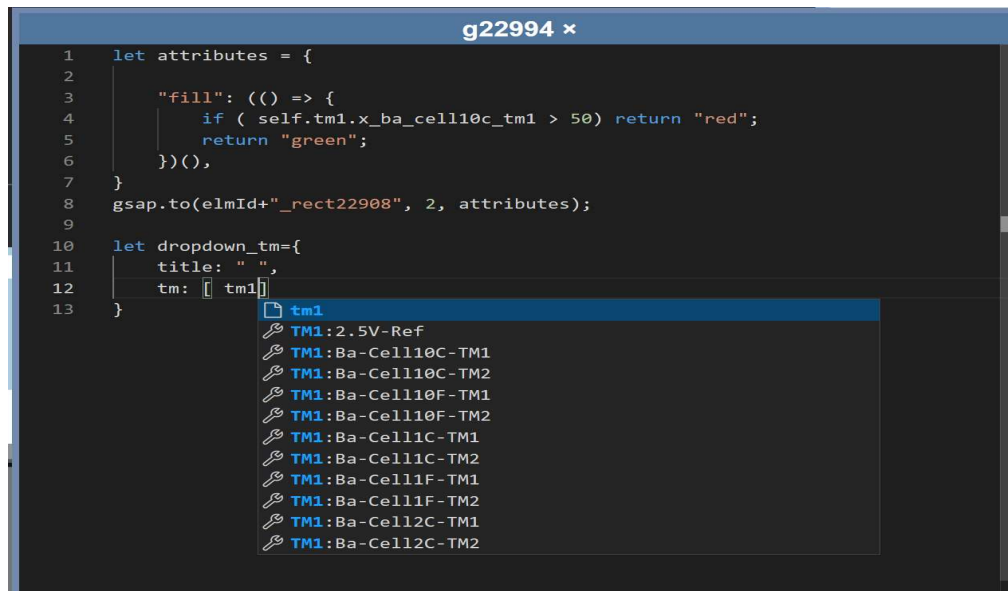


Figure 15: VS code editor for mapping element attributes to mnemonics

6.5 TM/SMON/ADC Simulator

This module is responsible for simulating TM/SMON/ADC data for validation of block diagrams made by the user. Below is the detailed flowchart of simulator module.

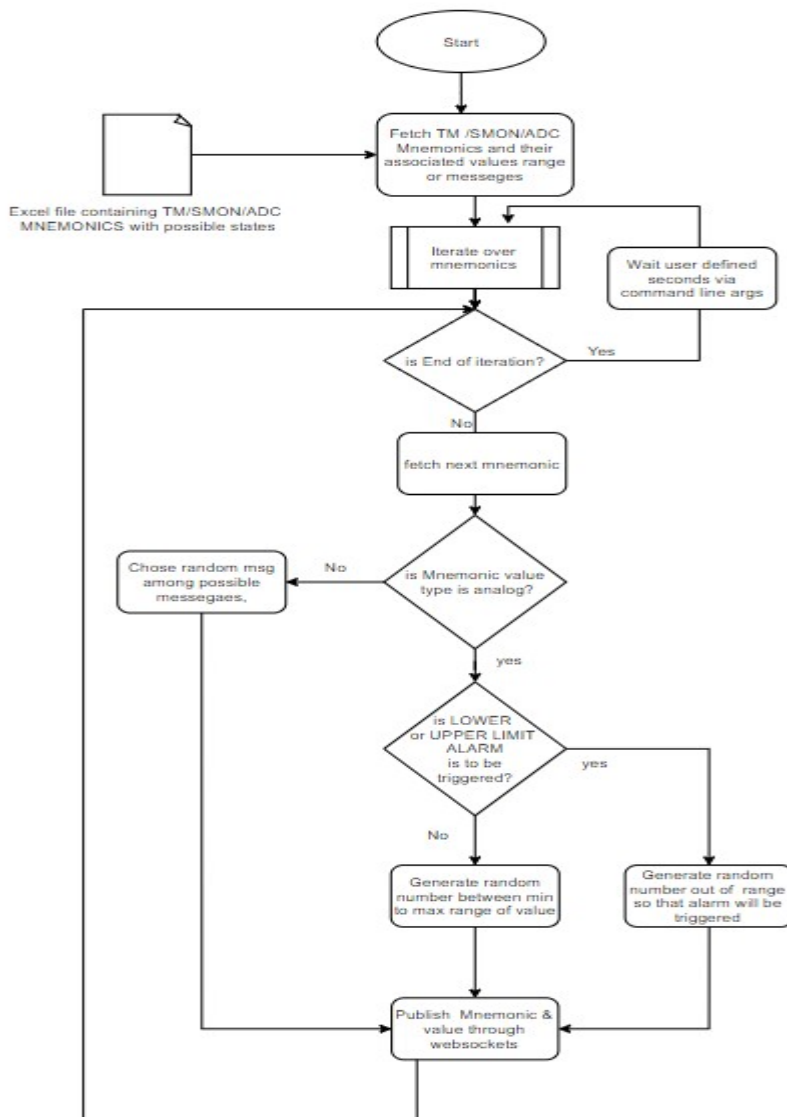


Figure 16 : Simulator Flowchart

On start from excel (Refer section 5.1) this module fetches all mnemonics and for each mnemonic min, max value or possible allotted messages. Then it iterates over these mnemonics, for each mnemonic a random telemetry value between min, max or a random choice among all possible messages will assigned and published through web sockets. If user wants to simulate limit crossing alarms for analog parameters, user has to enter 1 in excel sheet as described in section 5.1.



6.6 Client-side JavaScript

In client side VUE js frame work is used. Vue.js is a progressive JavaScript framework, which is used to build UIs (User Interfaces) and SPAs (Single-page Applications). This framework is famous for its fast-paced learning curve. It is such an easy to learn and approachable library that with the knowledge of HTML, CSS, and JavaScript, we can start building web applications in Vue.js. It is a versatile framework for our need as a library or a full-fledged framework for building huge web apps. In Vue, the bound data is updated reactively as are DOM objects, which is excellent for any application requiring real-time updates.

In addition to vue following JavaScript libraries are used for changing element attributes and to create animation effects.

1. GSAP: It is an industry standard JavaScript animation library that lets you craft high-performance animations that work in every major browser
2. Animejs: It is a lightweight JavaScript animation library with a simple, yet powerful API. It works with CSS properties, SVG, DOM attributes and JavaScript Objects.
3. Snap svg: It is a brand-new JavaScript library for working with SVG. Snap provides web developers with a clean, streamlined, intuitive, and powerful API for animating and manipulating both existing SVG content, and SVG content generated with Snap.

6.6.1 UI Thread functionality

On browser loading config file will be loaded which contents WAMP url and TM1, TM2, SMON1, SMON2 WebSocket urls. After successful loading of configuration file browser will connect to Wamp server and tm1, tm2 mnemonics will be posted to worker thread, the worker thread will initiate WebSocket subscription request to SCC. The UI thread on successful connection to Wamp server initiates RPC call to SPASDACS API to get vs code editor configuration details, after receiving the response from RPC call vs code editor will be initialized. And also, the UI thread will initialize vue JS application component. Inside vue JS on mounted hook events will be registered to handle display of vs code editor, builder option, dropdown display this finishes the loading process. After this whenever change is detected in data the corresponding vue js computed method will be invoked which causes update in UI.

6.6.2 TM/SMON/ADC Data Reception and pre-processing by Web Worker

To handle TM/SMON/ADC data reception and pre-processing a web worker is created to offload UI thread. In browser after SPASDACS page load, UI thread posts message to web worker



with list of TM Mnemonics to be subscribed from SCC. Web worker after receiving message from UI thread, it sends TM/SMON/TC sent subscription request to SCC server.

The subscription format is as described below

Telemetry Data Interface:

To subscribe/unsubscribe to telemetry the following request to be sent to web socket server

```
{"action": "subscribe/unsubscribe", "params": ["Frame-id", "OBT"] }
```

After subscription the telemetry is transmitted continuously to client based on sampling rate of the telemetry parameter.

TM Data message format:

```
{"param_id":"","param":"","source_info":"","raw_count":"","proc_value":"","time_stamp":"","upper_limit":"","lower_limit":"","err_desc":""}
```

Telecommand Data Interface:

To subscribe/unsubscribe to telecommand the following request to be sent to web socket server

```
{"action": "subscribe/unsubscribe"}
```

After subscription, whenever the telecommand is issued to spacecraft the information is transmitted to client in the following json format

```
{"cmd":"","code":"","full_code":"","data_part":"status":"","time":""}
```




SMON/ADC Data Interface:

To subscribe/unsubscribe to SMON/ADC data the following request to be sent to web socket server

```
{"action": "subscribe/unsubscribe"}
```

After subscription, every 4 seconds the information is transmitted to client in the following json format

```
{" paramlist": [{ "param": "", "value": "" }, { "param": "", "value": "" } ... ], "stream": "", "seqcount": "", "time": "", "error": "" }
```

In case of link failure between SCC to PCC the worker threads continuously keeps trying to re connect with 5 seconds gap between each attempt.

When data is received from SCC, the TM/SMON/ADC mnemonics will be renamed (all characters other than a-z & 0-9 will be replaced with underscore) to make them as valid JavaScript variables. As JavaScript variables should not start with numbers, “x_” will be added to all mnemonics.

Renaming Examples:

Sno.	Actual Mnemonic	Renamed Mnemonic
1	N-Bus-LD-Cur	x_n_bus_ld_cur
2	30V-Supply-Vol	x_30v_supply_vol
3	BCU1-O/P-Vol	x_bcu1_o_p_vol

Table 3: TM/SMON Mnemonics renaming examples

In addition to renaming the mnemonics this web worker will monitor upper & lower limits for all analog parameters, if parameter crosses it limits will be notified to UI thread which in turn displays alarm (voice & visual). Data break status will be posted to UI thread, which in turn displays



alarm (voice & visual). After renaming the mnemonics the received data will be posted to UI thread every 1 second. The UI thread assigns the received data to Vuejs data.

6.7 Inkscape Editor

Inkscape is a free and open source vector graphic editor used to create vector images, primarily in scalable vector graphics (svg) format. Other formats can be imported and exported.

Inkscape can render primitive vector shapes (e.g. rectangle, ellipses, polygons, arcs, spirals, stars and 3D boxes) and text. These objects may be filled with solid colors, patterns, radial or linear color gradients and their borders may be stroked, both with adjustable transparency. Embedding and optional tracing of raster graphics is also supported, enabling the editor to create vector graphics from photos and other raster sources. Created shapes can be further manipulated with transformations, such as moving, rotating, scaling and skewing.

Features:

Inkscape workflow is based around vector objects. Tools allow manipulating primitive vector shapes: simple ones like rectangles, ellipses and arcs, as well as more complex objects like 3D boxes with adjustable perspectives, stars, polygons and spirals. Rendering feature that can create objects like barcodes, calendars, grids, gears and roulette curves (using the spiro graph tool). These objects may be filled with solid colors, patterns, radial or linear color gradients and their borders may be stroked, both with adjustable transparency. All of those can be further edited by transformations—such as moving, rotating, scaling and skewing—or by editing paths. Other tools allow creating Bezier curves, freehand drawing of lines (pencil), or calligraphic (brush-like) strokes which support a graphics tablet.

Inkscape is able to write and edit text with tools available for changing font, spacing, kerning, rotation, flowing along the path or into a shape. Text can be converted to paths for further editing. The program also has a layers (as well as an objects) feature that allows the user to organize objects in a preferred stacking order in the canvas. Objects can be made visible/invisible and locked/unlocked through these features. Symbol libraries enable Inkscape to use existing symbols like logic-gate symbols or DOT pictograms. Additional libraries can be included by user. Inkscape supports image tracing, the process of extracting vector graphics from raster sources. Clones are child objects of an original parent object. Different transformations can be applied to them, such as: size, position, rotation, blur, opacity, color and symmetry. Clones are updated live whenever the parent objects changes.

Object manipulation:

Every object in the drawing can be subjected to arbitrary affine transformations: moving, rotating, scaling, skewing and a configurable matrix [clarification needed]. Transformation parameters can be specified numerically. Transformations can snap to angles, grids, guidelines and nodes of other objects, or be aligned in specified direction, spaced equally, scattered at random. Objects can be grouped together. Groups of objects behave similarly to objects. Objects in a group can be edited without having to ungroup them first. The Z-order determines the order in which



objects are drawn on the canvas. Objects with a high Z-order are drawn on top of objects lower in the Z-order. Order of objects can be managed either using layers, or by manually moving the object up and down in the Z-order. Layers can be locked or hidden, preventing modifying and accidental selection. The Create Tiled Clones tool allows symmetrical or grid-like drawings using various plane symmetries. Appearance of objects can be further changed by using masks and clipping paths, which can be created from arbitrary objects, including groups. The style attributes are 'attached' to the source object, so after cutting/copying an object onto the clipboard, the style's attributes can be pasted to another object.

6.7.1 Symbol library

Inkscape editor had a feature to add custom symbol library. The more frequently used shapes/diagrams can be created using inkscape editor and those shapes/diagrams can be added to separate svg file. The symbols svg file should be placed inside 'share\inkscape\symbols' folder this folder will be there inside inkscape installation directory. After placing the file inside the folder specified those symbols can be used inside editor.

6.8 Detailed Design Description

API Reference

This section contains auto-generated API reference documentation.

src

Subpackages

src.Core

Subpackages

src.Core.EditorDataAPI

Submodules

src.Core.EditorDataAPI.AttrDefinations

Module Contents

src.Core.EditorDataAPI.AttrDefinations.attrs

src.Core.EditorDataAPI.EditorData

Module Contents

Classes

EditorData This class is responsible for creating necessary data to configure VS code editor. It will inject all TM/SMON/ADC

**class src.Core.EditorDataAPI.EditorData.EditorData**

Bases: src.Core.userAuthentication.UserAuthentication

This class is responsible for creating necessary data to configure VS code editor. It will inject all TM/SMON/ADC

mnemonics inside editor config file for search and auto completion of mnemonics inside VS code editor.

get_editor_config(self, user_details: dict = {})

This function gets all mnemonics and creates VS code configuration file.

Parameters

user_details (*dict*) – user credentials {username:"",password:""}

Returns

JSON formatted configuration file

Return type

string

save_map_data(self, element_id, user_defined_attribute_mapping_js_code, error_status, html_page_title, user_details={})

This function is responsible for saving user defined attributes by using VS code editor into file

Parameters

- **element_id** (*string*) – element ID of html/svg markup
- **user_defined_attribute_mapping_js_code** (*string*) – attribute map data JavaScript code
- **error_status** (*string*) – JavaScript code syntax status
- **html_page_title** (*string*) – page title
- **user_details** (*dict*) – user credentials {username:"",password:""}

Returns

Status Saved/Not Allowed to Save

Return type

string

get_map_data(self, element_id, html_page_title, user_details={})

This function is responsible for reading user defined attributes.

Parameters

- **element_id** (*string*) – element ID of html/svg markup



- **html_page_title** (*string*) – page title
- **user_details** (*dict*) – user credentials {username:"",password:""}

Returns

mapdata/Not Allowed to read

Return type

string

src.Core.database

Submodules

src.Core.database.attributeStore

Module Contents**Classes**

AttribDb This class is responsible for store and retrieve data from mapdata.json which contains all elements

Attributes

file_dir

path

file_path

map_data

src.Core.database.attributeStore.file_dir

src.Core.database.attributeStore.path

src.Core.database.attributeStore.file_path

src.Core.database.attributeStore.map_data

class src.Core.database.attributeStore.AttribDb

This class is responsible for store and retrieve data from mapdata.json which contains all elements attribute mapping code defined by user

get_local_store(self)

set_local_store(self, key, value)

reset_local_store(self)

get_vue_variables_and_tm_names(self)

This function telemetry mnemonics used in all element id attribute mapping data.

**Returns**

{“tm1”:self._vue_variables_tm1, “tm2”:self._vue_variables_tm2}, self._tm1_names,
self._tm2_names

Return type

Dict,list,list

get_variables(*self, elmid_map, tm_chain*)

This function extracts all mnemonics related given telemetry chain used in attribute data map defined by user.

Parameters

- **elmid_map** (*string*) – Mapdata JavaScript code
- **tm_chain** (*string*) – Telemetry chain tm1/tm2

Returns

TM mnemonics list

Return type

list of strings

get_element_data_map(*self, elementID*)

This function reads user defined code for the element when Ctrl+click event is triggered on GUI.

Parameters

elementID –

Returns**Return type****store_value_to_file(*self, element_id, attribute_map_code, error_status, web_page_title*)**

This function stores user defined code on GUI VS code editor when save is triggered.

Parameters

- **element_id** (*string*) – Html/svg element ID
- **attribute_map_code** (*string*) – user defined attribute_map_code from VS code editor
- **error_status** (*string*) – attribute_map_code syntax status
- **web_page_title** (*string*) – web page title

Returns

None

Return type

None



src.Core.database.getMnemonics

Module Contents

Classes

GetMnemonics This class is responsible for reading all TM/SMON/ADC mnemonics from data.xlsx excel sheet.

Attributes

file_dir

path

excel_path

src.Core.database.getMnemonics.file_dir

src.Core.database.getMnemonics.path

src.Core.database.getMnemonics.excel_path

class src.Core.database.getMnemonics.GetMnemonics

This class is responsible for reading all TM/SMON/ADC mnemonics from data.xlsx excel sheet.

get_tm_mnemonics(*self*)

rename_mnemonic(*self*, *mnemonic*)

This function renames the mnemonics to make them valid JavaScript variables by removing special characters

Parameters

mnemonic (*string*) – Actual Mnemonic

Returns

Renamed Mnemonic

Return type

string

get_tm_new_name_actual_name_map(*self*)

This function generates dictionary of all telemetry mnemonics with modified_name as key and original name as value

Returns

tm_newName_actualName_map

Return type

dict

**get_tm_mnemonics_fromDB(self)**

This function reads all telemetry mnemonics from pandas dataframe which contains data.xlsx TM sheet

Returns

tm_newName_actualName_map,renamed_mnemonic_list

Return type

dict,list of strings

get_smon_mnemonics_fromDB(self)

This function reads all telemetry mnemonics from pandas dataframe which contains data.xlsx SMON sheet

Returns

smon_newName_actualName_map,renamed_mnemonic_list

Return type

dict,list of strings

get_smon_data(self)

This function returns smon data

Returns

smon_newName_actualName_map,renamed_mnemonic_list

Return type

dict,list of strings

src.Core.svgComponents

Submodules

[src.Core.svgComponents.SVG_Parser](#)

Module Contents

Classes

SVGParser This class is responsible to fetching all elements in the given svg file and calls appropriate apis to generate vue

Attributes

logger

src.Core.svgComponents.SVG_Parser.logger

class src.Core.svgComponents.SVG_Parser.SVGParser

Bases: **src.Core.svgComponents.componentFactory.ComponentAPI**



This class is responsible to fetching all elements in the given svg file and calls appropriate apis to generate vue computed methods and dropdown tm/smon/adc data table html markup

get_svg_layer_names(*self*, *inputFile*)

This Function gets the label names of all layers in a svg file. And makes a dictionary with label of layer as key and its id as value

Parameters

inputFile (*string*) – SVG File absolute path

Returns

layer_to_label_id_dict, layer_labels

Return type

Dict

get_svg_page_colors(*self*, *inputFile*)

This function reads svg file background color.

Parameters

inputFile (*string*) – svg file name

Returns

svg file background color in hex(#rrggbb) color code

Return type

string

editSvg(*self*, *inputFile*, *file_name*)**remove_unsupported_text(*self*, *svg_markup*)**

This function removes the markup (generated by inkscape editor) that is not supported by browser.

Parameters

svg_markup (*string*) – svg markup

Returns

svg markup

Return type

string

[src.Core.svgComponents.componentFactory](#)

Module Contents

Classes



ComponentAPi

Attributes

custom

user

src.Core.svgComponents.componentFactory.custom

src.Core.svgComponents.componentFactory.user

class src.Core.svgComponents.componentFactory.ComponentAPi

classmethod replaceAndBindProps(cls, et_root, elementId, component_name, file_name)

This function invokes appropriate api based on element ID to generate required vue computed methods and also html markup for dropdown html will be generated.

Parameters

- **et_root** (*xml etree ref*) – xml etree ref of svg file
- **elementId** (*string*) – element ID
- **component_name** (*string*) – component name
- **file_name** (*string*) – svg file name

Returns

computed_props, drop_down_tm,el_id

Return type

string,string,string

classmethod get_component_attrb_format(cls, component_name)

This function fetches gui attribute map data template for the given component type

Parameters

component_name (*string*) – Name of the component

Returns

attribute map data template

Return type

string

[*src.Core.svgComponents.component_customComponent*](#)

Module Contents

Classes



CustomComponent

class src.Core.svgComponents.component_customComponent.CustomComponent

Bases: src.Core.svgComponents.replaceBlocksBase.ReplaceBlocksBase

replace_with_block(self, root, id_to_replace)

This function is responsible to replace use element markup with markup defined in replaceBlocks.svg file and changes element id by replacing - and space with underscores.

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference
- **id_to_replace** (*string*) – original element id

Returns

final element id

Return type

string

change_ids_recursively(self, id, element)

This function changes element ids of all child elements of the given element

Parameters

- **id** (*string*) – element ID
- **element** (*element tree ref*) – xml element tree ref

Returns

None

Return type

None

attach_props(self, root, element_id)

This function is responsible to create compute methods for each element. And also generates tm dropdown html markup.

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference
- **element_id** (*string*) – element id

Returns

None

Return type

None

**get_computed_props(*self*, *element_id*)**

This function return computed method code and drop down html markup

Parameters

element_id (*string*) – element Id

Returns

computed method code and dropdown html markup

Return type

string,string

[src.Core.svgComponents.component_userComponent](#)

Module Contents

Classes

UserComponent

Attributes

component_name

src.Core.svgComponents.component_userComponent.component_name = user

class src.Core.svgComponents.component_userComponent.UserComponent

Bases: src.Core.svgComponents.replaceBlocksBase.ReplaceBlocksBase

replace_with_block(*self*, *root*, *id_to_replace*)

This function is responsible to replace use element markup with markup defined in replaceBlocks.svg file and changes element id by replacing - and space with underscores.

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference
- **id_to_replace** (*string*) – original element id

Returns

final element id

Return type

string

attach_props(*self*, *root*, *element_id*)

This function is responsible to create compute methods for each element. And also generates tm dropdown html markup.

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference



- **element_id** (*string*) – element id

Returns

None

Return type

None

get_computed_props(self, element_id)

This function return computed method code and drop down html markup

Parameters

element_id (*string*) – element Id

Returns

computed method code and dropdown html markup

Return type

string,string

[*src.Core.svgComponents.replaceBlocksBase*](#)

Module Contents

Classes

ReplaceBlocksBase

Attributes

file_dir

src.Core.svgComponents.replaceBlocksBase.file_dir

class src.Core.svgComponents.replaceBlocksBase.ReplaceBlocksBase

static read_component_code_block(component_name)

The function reads pre defined GUI data mapping code template based on component name.

Parameters

component_name (*string*) – component name

Returns

GUI data mapping code template

Return type

string

classmethod blockAttrMap(cls, component_name=None)

get_predefined_JavaScript_code(self, component_name)

The function reads pre defined JavaScript code for assigning attribute to elements

**Parameters**

component_name (*string*) – component name

Returns

JavaScript code

Return type

string

***static* get_predefined_common_JavaScript_code()**

The function return JavaScript code which is common to all elements.

Returns

JavaScript code

Return type

string

***static* get_predefined_common_JavaScript_gui_code()**

The function return pre defined GUI data mapping code which is common to all elements.

Returns

GUI data mapping code template

Return type

string

extract_tm_drop_down_data_from_code(self, gui_attribute_mapping_code)

This function extracts dropdown telemetry details from the user defined attribute data mapping.

Parameters

gui_attribute_mapping_code (*string*) – User mapping data

Returns

dict {"title": title, "mnemonics": mnemonics_comma separated },rest of the mapping code

Return type

dict,string

get_element_initial_attributes(self, element_ref)

This function reads all initial attributes exists on a given svg element

Parameters

element_ref (*xml element tree reference*) – xml element tree reference of the element

Returns

dictionary with attributes as key and attribute value as value

**Return type**

Dict

get_style_dict(*self*, *style*)

This function generates dictionary out of css style string.

Parameters

style (*string*) – css style string

Returns

dictionary with style type as key and style value as value

Return type

dict

get_use_element_ids(*self*, *root*, *prefix*='{<http://www.w3.org/2000/svg>}')

This functions fetches all elements id of element tag type 'use'

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference
- **prefix** (*string*) – svg standard defnaton '<http://www.w3.org/2000/svg>'

Returns

element ids of all tags with 'use'

Return type

list of strings

add_gradients(*self*, *input_file_svg_root*)

This function adds all defs section of predefined replaceBlocks.svg file to the given input svg file.

Parameters

input_file_svg_root (*xml etree element reference*) – input svg xml etree element reference

Returns

None

Return type

None

abstract_replace_with_block(*self*, *root*, *id_to_replace*)

This function is responsible to replace use element markup with markup defined in replaceBlocks.svg file and changes element id by replacing - and space with underscores.

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference



- **id_to_replace** (*string*) – original element id

Returns

final element id

Return type

string

abstract attach_props(self, root, element_id)

This function is responsible to create compute methods for each element. And also generates tm dropdown html markup.

Parameters

- **root** (*xml etree element reference*) – input svg xml etree element reference
- **element_id** (*string*) – element id

Returns

None

Return type

None

abstract get_computed_props(self, element_id)

This function return computed method code and drop down html markup

Parameters

element_id (*string*) – element Id

Returns

computed method code and dropdown html markup

Return type

string,string

replace_block_api(self, input_file_svg_root, elm_id, file_name="", component_name=None)

This function invokes appropriate child class function to generate computed method code.

Parameters

- **input_file_svg_root** (*xml etree element reference*) – input svg xml etree element reference
- **elm_id** (*string*) – id of the svg element
- **file_name** (*string*) – svg filename
- **component_name** (*string*) – svg component name

**Returns**

_computed_props, _drop_down_tm, elm_id

Return type

list of strings

src.Core.utility

Submodules

src.Core.utility.colors

Module Contents**Classes**

Colors

class *src.Core.utility.colors.Colors*

get_led_block_color_map(*self*)

get_color_map(*self*)

src.Core.utility.generalFunctions

Module Contents**Classes**

GeneralFunctions

class *src.Core.utility.generalFunctions.GeneralFunctions*

get_style_dict(*self, element*)

This function fetches style attributes for the given svg element

Parameters

element (*xml etree ref*) – element ref

Returns

style attributes as dictionary

Return type

dict

get_style_string_from_dict(*self, styleDict*)

This function converts style dictionary to css style string

Parameters

styleDict (*dict*) – style attributes dictionary

**Returns**

style css string

Return type

string

get_camle_case(*self*, *string*)

This function converts given string to camelCase convention.

Parameters

string (*string*) – input string

Returns

camelCase string

Return type

string

[*src.Core.utility.getDropDownTm*](#)

Module Contents

Classes

GetDropDownTm This class is responsible for generating drop down telemetry table HTML mark up

class *src.Core.utility.getDropDownTm.GetDropDownTm*

This class is responsible for generating drop down telemetry table HTML mark up

get_original_mnemonic(*self*, *renamed_mnemonic*)

This function searches 'tm_renamed_to_actual_name' dictionary for a given renamed_mnemonic and returns actual_mnemonic

Parameters

renamed_mnemonic –

Returns

tm_mnemonic

Return type

string

get_tm_dropdown_html(*self*, *dropdown_tm*, *element_id*)

This function generates HTML markup dropdown telemetry

Parameters

- **dropdown_tm** (*dict*) – {"title":"","mnemonics":""}



- **element_id** (*string*) – element ID

Returns

HTML Markup

Return type

string

[src.Core.utility.registerSvgNameSpace](#)*Module Contents*

```
src.Core.utility.registerSvgNameSpace.xlink = {http://www.w3.org/1999/xlink}href
src.Core.utility.registerSvgNameSpace.prefix = {http://www.w3.org/2000/svg}
```

[src.Core.utility.singleTonMeta](#)*Module Contents**Classes*

Singleton

class src.Core.utility.singleTonMeta.Singleton

Bases: type

_instances**__call__(cls, *args, **kwargs)**

Call self as a function.

[Submodules](#)[src.Core.API](#)*Module Contents**Classes*

loggingHandler This class configures logging module.And publishes all logs to WAMP server.

Spasdacs This Class registers all the smart TC API functions with WAMP server.

Functions

runSpasdacs()

Attributes

file_handler



logger

src.Core.API.file_handler

src.Core.API.logger

class src.Core.API.loggingHandler(*_this*)

Bases: logging.StreamHandler

This class configures logging module. And publishes all logs to WAMP server.

emit(*self*, *record*)

Emit a record.

If a formatter is specified, it is used to format the record. The record is then written to the stream with a trailing newline. If exception information is present, it is formatted using `traceback.print_exception` and appended to the stream. If the stream has an 'encoding' attribute, it is used to determine how to do the output to the stream.

class src.Core.API.Spasdacs(*config=None*)

Bases: autobahn.asyncio.wamp.ApplicationSession,

src.Core.EditorDataAPI.EditorData.EditorData, src.Core.Main.CreateSpasDacsApp

This Class registers all the smart TC API functions with WAMP server.

async onJoin(*self*, *details*)

This function gets called once the connection with WAMP server is successful. And this will registers all the smart TC API functions with WAMP server.

Parameters

details (*autobahn.wamp.types.SessionDetails*) –

Returns

Return type

Union[_asyncio.Future, None]

src.Core.API.runSpasdacs()

[src.Core.Main](#)

[Module Contents](#)

Classes

CreateSpasDacsApp This class is responsible for creating web application based on inputfiles and user mapping data.

Attributes

logger

file_dir



path

config_file_path

src.Core.Main.logger

src.Core.Main.file_dir

src.Core.Main.path

src.Core.Main.config_file_path

class src.Core.Main.CreateSpasDacsApp

Bases: src.Core.userAuthentication.UserAuthentication,

src.Core.WriteHtml.WriteHtml, src.Core.writeJavaScript.WriteJavaScript,

src.Core.writeCss.WriteCss, src.Core.svgComponents.SVG_Parser.SVGParser

This class is responsible for creating web application based on inputfiles and user mapping data.

init(self)

read_nav_bar_right_template(self)

This reads predefined html template for navbar right side elements. :return: html markup

:rtype: string

make_html_navbar(self, page_no, page_back_ground_color)

This function generates required navbar for the web page.

Parameters

- **page_no** (*int*) – web page number
- **page_back_ground_color** (*string*) – color hex code

Returns

Html markup

Return type

string

get_input_files_ordered(self, file_directory_path)

This Function reads all svg files inside 'file_directory_path' and creates an array of all svg file names and a dictionary with file name as key and its absolute path as value. If user had given file names as xyz_1.svg, sdg_2.svg etc i.e "filename_i.svg" i is from 1 to n. Then this function order the file names array from 1 to n.

Example 1:

Directory has following files a_5.svg, b_2.svg, c_1.svg, d.svg, e.svg then file array = [c_1.svg, b_2.svg, a_5.svg, d.svg, e.svg]

Example 2:



Directory has following files a.svg,b.svg,c.svg,d.svg,e.svg then file array =
[a.svg,b.svg,c.svg,d.svg,e.svg]

Parameters

file_directory_path (*string*) – svg files directory path

Return ordered_files, file_paths**Return type**

List of [List of Strings , Dictionary]

pseudo code:

```
>>>
    Fetch all svg files in directory
    if File_name has priority:
        order files as per priority
```

**get_nav_links(self, ordered_file_names: Array of file names,
file_name_to_absolute_file_path_dict: Dictionary {filename: abs_path})** → List of
[layer_names, array of laer_to_label_id_dict]

This function gets all the layer names of input svg files. These names will be used in navigation dropdown menu for selecting different pages in UI.

Parameters

- **ordered_file_names** (*List of String*) – svg file names
- **file_name_to_absolute_file_path_dict** (*Dict*) – absoulute file paths

Returns

layer names, layer_to_label_id_dict_array

Return type

List of List of Strings , Dict

generate_contrast_color(self, background_color, bias=30)

This function generates biased color for give input color

Parameters

- **background_color** (*string*) – color hex value
- **bias** (*int*) – offset to be applied

Returns

color hex value

Return type

string

**create_spasdacs_app(*self*, *user_details*: dict = {})**

This is entry point for spasdacs build functionality. This function will call all relevant function to generate final web application if user have desired permissions.

Parameters

user_details (*dict*) – {"user_name": "", "password": ""}

Returns

build success status

Return type

string

src.Core.WriteHtml

Module Contents

Classes

WriteHtml

Attributes

file_dir_html

src.Core.WriteHtml.file_dir_html

class src.Core.WriteHtml.WriteHtml

createHtmlFile(*self*, *distribution_directory_path*, *file_name*, *original_file_name*, *nav_bar*, *svg_content*, *drop_down_tm_html*, *background_color*)

This function creates html file for the given input svg file.

Parameters

- **distribution_directory_path** (*string*) – path of web deployment folder
- **file_name** (*string*) – html file name
- **original_file_name** (*string*) – input svg file name
- **nav_bar** (*string*) – html markup for navbar
- **svg_content** (*string*) – svg markup
- **drop_down_tm_html** (*string*) – html markup for drop_down_tm
- **background_color** (*string*) – Html page background color

Returns

None

**Return type**

None

read_predefined_html_template(*self*)

This function reads the predefined html template.

Returns

Html template

Return type

string

read_predefined_js_files_template(*self*)

This function reads the predefined script tags defined for loading js files

Returns

Html markup

Return type

string

html(*self*, *title*, *nav_bar*, *svg_content*, *drop_down_tm_html*, *file_name*)

This function takes html template and substitutes the data specific to the given input file.

Parameters

- **title** (*string*) – Html page title
- **nav_bar** (*string*) – html markup for navbar
- **svg_content** (*string*) – svg markup
- **drop_down_tm_html** (*string*) – html markup for drop_down_tm
- **file_name** (*string*) – html file name

Returns

Html markup

Return type

string

saveHtmlFile(*self*, *distribution_directory_path*, *html*, *file_name*)

The function saves the generated html file in web deployment folder

Parameters

- **distribution_directory_path** (*string*) – file path
- **html** (*string*) – html markup



- **file_name** (*string*) – html file name

Returns

None

Return type

None

src.Core.Logger_config

Module Contents

Functions

`get_logger(log_file_name, path='C:/Users/Public/webapps_logs')`

This function creates rotating file logger. When log file size crosses 100KB it overwrites the log file.

`get_logger_handler(log_file_name, path='C:/Users/Public/webapps_logs')`

This function creates rotating file logger. When log file size crosses 100KB it overwrites the log file.

Attributes

`log_format`

`src.Core.logger_config.log_format = %(asctime)s %(levelname)s`

`%(name)s: %(funcName)s(%(lineno)d): %(message)s`

`src.Core.logger_config.get_logger(log_file_name, path='C:/Users/Public/webapps_logs')`

This function creates rotating file logger. When log file size crosses 100KB it overwrites the log file.

Parameters

- **log_file_name** (*string*) – name of the log file
- **path** (*string*) – Log file path

Returns

logger object

Return type

object

`src.Core.logger_config.get_logger_handler(log_file_name,`

`path='C:/Users/Public/webapps_logs')`

This function creates rotating file logger. When log file size crosses 100KB it overwrites the log file.

**Parameters**

- **log_file_name** (*string*) – name of the log file
- **path** (*string*) – Log file path

Returns

logger handler object

Return type

object

[*src.Core.login_module*](#)

Module Contents

Classes

UserAccounts

Attributes

file_dir_credentials

obj

src.Core.login_module.file_dir_credentials

class src.Core.login_module.UserAccounts

read_credentials(self)

This function reads user credential from the file

Returns

None

Return type

None

save_credentials(self)

This function saves user credential to a file

Returns

None

Return type

None

check_credentials_validity(self)

This function validates user credentials against save data in the file.

**Returns**

Credentials validity True/False

Return type

bool

create_super_user(*self*, *user_name*, *password*)

This function creates admin user.

Parameters

- **user_name** (*string*) – user_name
- **password** (*string*) – password

Returns

Status created/failed

Return type

string

src.Core.login_module.obj

src.Core.userAuthentication

*Module Contents**Classes*

UserAuthentication

class src.Core.userAuthentication.UserAuthentication**register_user(*self*, *user_details*: dict)**

This function registers new user

Parameters

user_details (*dict*) – {username:"",password:""}

Returns

status

Return type

string

remove_user(*self*, *user_details*: dict)

This function deletes registers user

Parameters

user_details (*dict*) – {username:"",password:""}

**Returns**

status

Return type

string

change_user_type(self, user_details: dict)

This function changes user type

Parameters**user_details** (*dict*) – {username:"",password:""}**Returns**

status

Return type

string

validate_user(self, user_details: dict)

This function validates user credentials

Parameters**user_details** (*dict*) – {username:"",password:""}**Returns**

validation status True/False

Return type

bool

is_build_allowed_by_user(self, user_details: dict)

This function checks whether the logged in user has authorization to build the application.

Parameters**user_details** (*dict*) – {username:"",password:""}**Returns**

validation status True/False

Return type

bool

[src.Core.writeCss](#)[Module Contents](#)[Classes](#)



WriteCss

Attributes

logger

src.Core.writeCss.logger

class src.Core.writeCss.WriteCss

**createCssFile(*self*, *distribution_directory_path*, *css_file_name*,
background_color='#101214')**

This function creates css file for the given input svg file

Parameters

- **distribution_directory_path** (*string*) – path of web deployment folder
- **css_file_name** (*string*) – css file name
- **background_color** (*string*) – web page background color

Returns

None

Return type

none

save_css_file(*self*)

This function saves the generated css file in the web deployment folder

:return:None :rtype:None

write_style(*self*, *background_color*='#101214')

This function adds css markup to change background color for the web page

Parameters

background_color (*string*) – svg file background_color

Returns

None

Return type

None

[*src.Core.writeJavaScript*](#)

Module Contents

Classes



WriteJavaScript This class is responsible for generating JavaScript file for the given svg file.

Attributes

logger

file_dir_js

src.Core.writeJavaScript.logger

src.Core.writeJavaScript.file_dir_js

class src.Core.writeJavaScript.WriteJavaScript

This class is responsible for generating JavaScript file for the given svg file.

createJavaScriptFile(*self, configuration, vue_variables, active_nav_link, tm1_names, tm2_names, computed_props, dropdown_tm_ids, index_file_name, org_file_name*)

This function generates JavaScript file for the input svg file

Parameters

- **configuration** (*Dict*) – {"distribution_directory_path": "path of web deployment folder", "file_name": "input svg file name"}
- **vue_variables** (*Dict*) – telemetry mnemonics defined in user mapping data
- **active_nav_link** (*int*) – GUI Navbar link id
- **tm1_names** (*array of strings*) – Chain 1 Telemetry Mnemonics to subscribe. Extracted from user mapping data
- **tm2_names** (*array of strings*) – Chain 2 Telemetry Mnemonics to subscribe. Extracted from user mapping data
- **computed_props** (*string*) – user defined data mapping attributes plus predefined js code for all componets in the given svg file.
- **dropdown_tm_ids** (*dict*) – ids of all TM drop down elements
- **index_file_name** (*string*) – Web page index file name
- **org_file_name** (*string*) – Input svg file name

Returns

None

Return type

None

get_smon_mnemonic_vue_variables(*self*)

This function creates vue data initializations for all SMON Mnemonics.

**Returns**

Array of smon1 & smon2 data dictionary

Return type

[Dict, Dict]

read_vue_mixin_js_code(*self*)

This function reads vue mixin code pre defined in vue_mixins.js file

Returns

vue mixin JavaScript code

Return type

string

read_vue_app_js_code(*self*)

This function reads vue initialization code pre defined in app.js file

Returns

vue initialization JavaScript code

Return type

string

read_global_scope_js_code(*self*)

This function reads web page global variables and initialization pre defined in global_scope.js file

Returns

web page global scope JavaScript code

Return type

string

save_file(*self*)

This function saves the generated JavaScript file in web app distribution folder.

Returns

None

Return type

None

define_global_variables(*self*)

This function inserts all global scope data of js file into JavaScript file

Returns

None

**Return type**

None

write_vue_app_code(*self*, *computed_props*)

This function assembles final code of the complete JavaScript file for the given svg file.

Parameters

computed_props (*string*) – user defined data mapping attributes plus predefined js code for all components in the given svg file.

Returns

None

Return type

None

6.9 Interface with Database

SPASDACS requires database interaction only during mapping of element attributes to TM/SMON/ADC mnemonics. The Data requirement is addressed in section 5.1

6.10 Interface with Spacecraft Checkout Computer (SCC)

SPASDACS app is interfaced with SCC by web sockets. When app loads in browser, subscription request for TM/SMON/ADC data will go to SCC server. After that SCC server will continuously send data to web browser.



7 Human Interface Design

7.1 Overview of user interface

UI for SPASDACS is a web page. On top menu bar it will have page selection drop down menu for switching between pages, Login & logout option, and Build button. User has to select page from page selection dropdown menu to change the page.

Modes of Operation in SPASDACS

1. TM view mode: - In this mode user can see the live telemetry in SPASDACS. This mode of operation is allowed to every user.
2. Programming mode: - In this mode previous mode features are allowed and also user can do element attributes mapping. Only Project Manager is allowed to operate SPASDACS in this mode.

7.2 Screen Images

The following snapshot shows the GUI of SPASDACS application, when mouse is hovered on any element the Telemetry dropdown will appear.

App build button Page Selection Dropdown TM/SMON Link Status

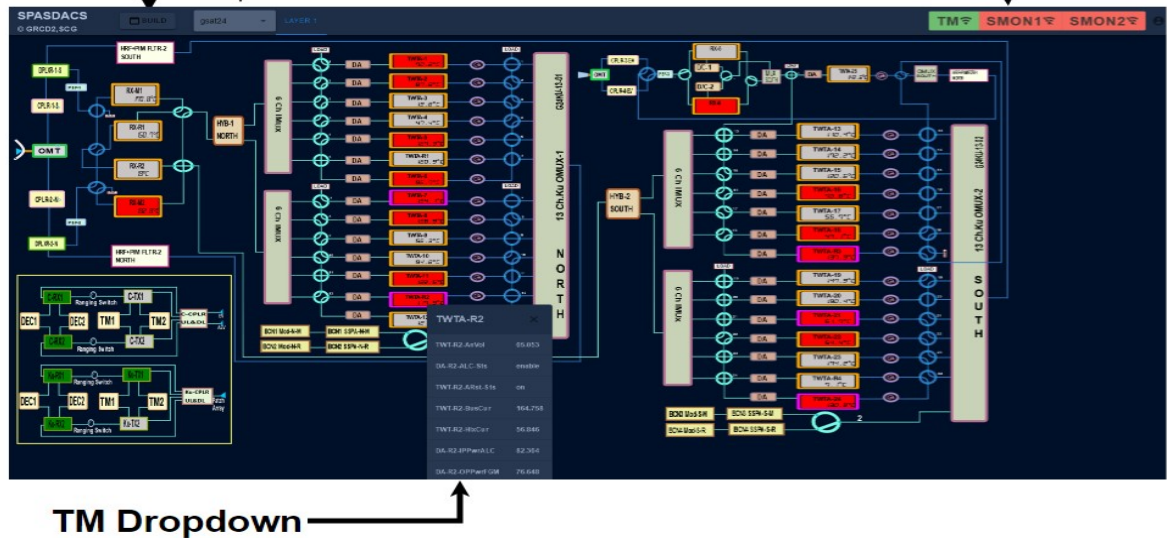


Figure 17: SPASDACS GUI-1

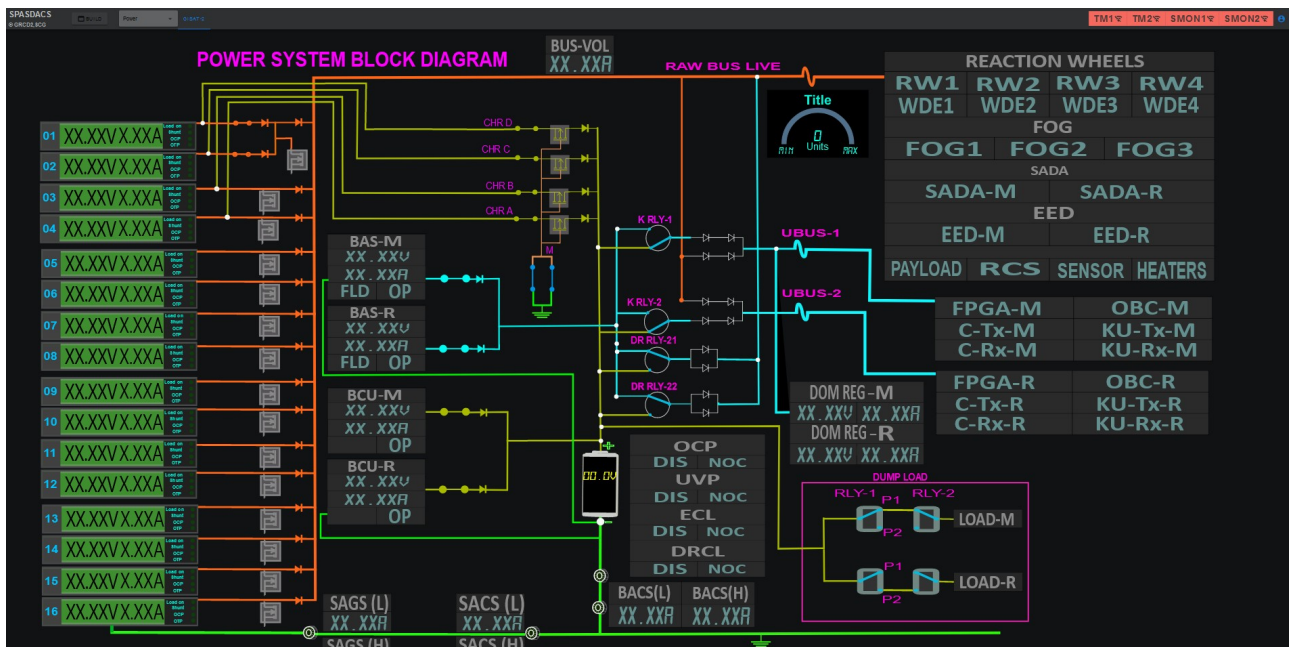


Figure 18: SPASDACS GUI-2

When user clicks on any elements by holding control key, VS code editor will appear as shown below. Inside this editor user has to assign TM/TC/SMON mnemonics

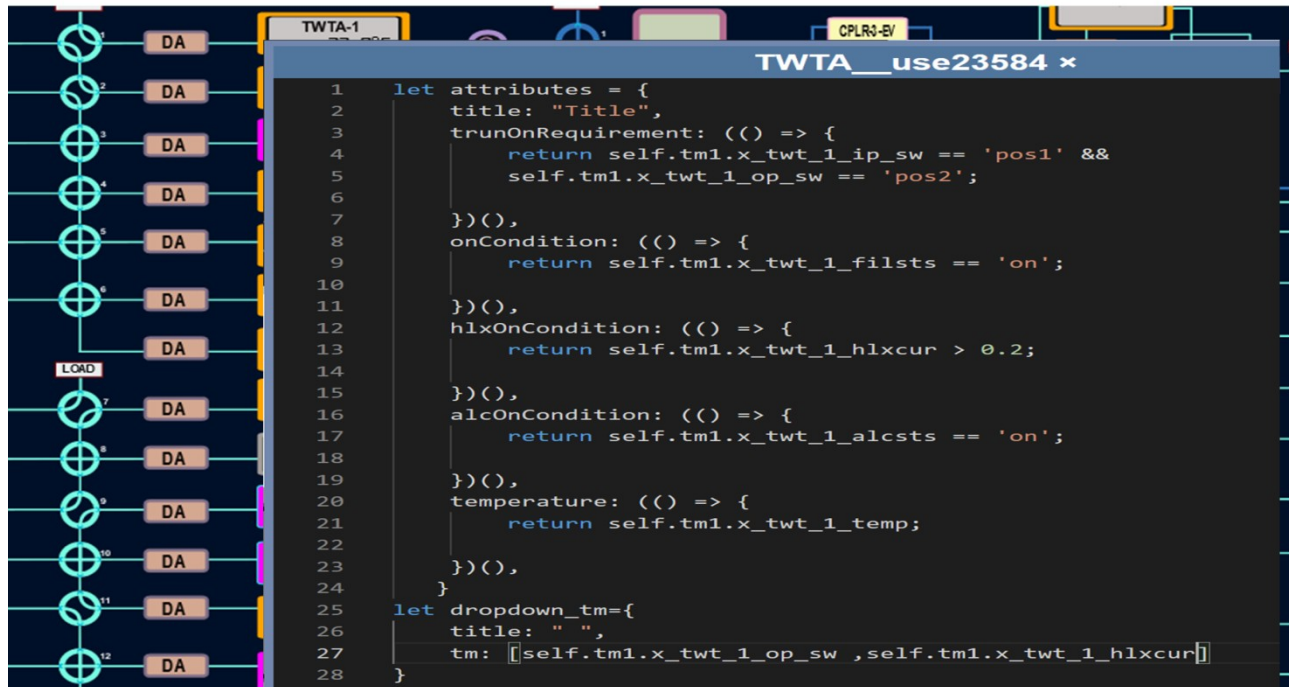


Figure 19: Dropdown VS code Editor for assigning TM/TC

Inside VS code editor automatic mnemonics dropdown will come when user starts typing the mnemonic as shown below

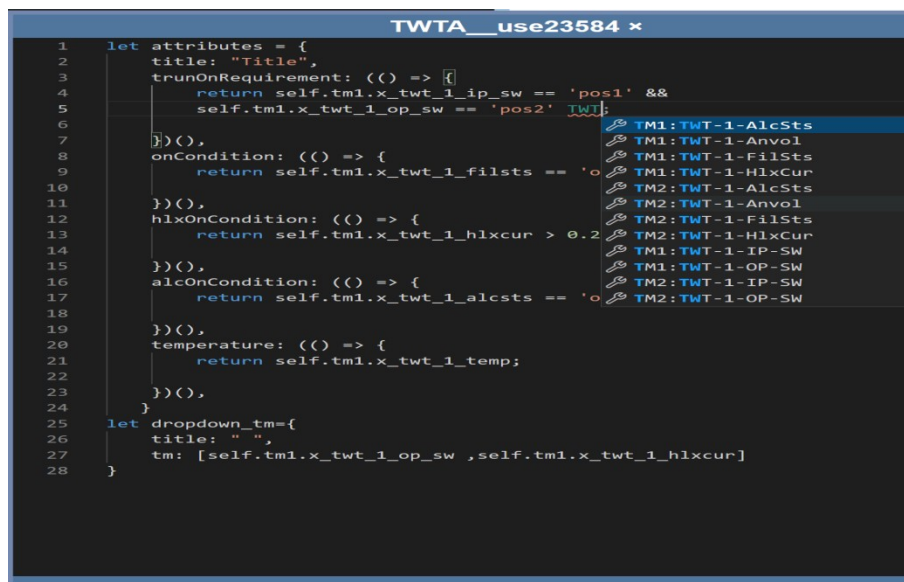


Figure 20: Mnemonic dropdown inside VS code editor

8 Requirements Matrix

Software Requirement Id	Section	Description
RID 1	6.7	Utility to create system block/representational diagram
RID 2	6.4	GUI to associate TM/SMON/ADC data.
RID 3	7.2	GUI to display TM/SMON/ADC real time data status
RID 4	6.7.1	Provision to create block diagram library
RID 5	6.1.2, 6.1.3	Linking TM/SMON/ADC data to Visual attribute of blocks
RID 6	6.6.2	Alerts on TM/SMON/ADC Data Break
RID 7	6.1.2, 6.1.3	Invalid condition/status of the blocks
RID 8	6.6.1	System Status Change Indication
RID 9	6.1.2	Telemetry Dropdown
RID 10	6.1	Multiple display pages
RID 11	3.1	The display shall be viewable on any PC over a LAN
RID 12	6.5	Telemetry/SMON/ADC data Simulator

Table 4 Requirements Matrix